Демидовский университет

Работа с файлами

Автор: доцент, кандидат технических наук
Ксения Владимировна Лагутина

Класс QIODevice

- QIODevice – базовый класс всех устройств ввода-вывода в Qt
- QIODevice предоставляет как общую реализацию, так и абстрактный интерфейс для устройств, поддерживающих чтение и запись данных
  - QFile, QBuffer и QTcpSocketopen() – открыть в заданном режиме OpenMode
- write(), putChar() – запись данных
- read(), readLine(), readAll() – чтение данных
- close() – закрыть устройство
- readyRead() – испускается, когда новые данные доступны для чтения

https://doc.qt.io/qtforpython-6/PySide6/QtCore/QIODevice.html

Демидовский университет

Класс QIODevice: типы устройств

- Устройства с произвольным доступом
  - seek() – поиск произвольных позиций
  - pos() – текущая позиция в файле
  - QFile и QBuffer
- Последовательные устройства не поддерживают поиск произвольных позиций
  - Данные должны быть прочитаны за один проход
  - QTcpSocket и QProcess
- isSequential() – тип устройства

Демидовский университет

Класс QDataStream

- QDataStream обеспечивает сериализацию двоичных данных в QIODevice
- Поток данных — это двоичный поток закодированной информации
- Можно использовать QDataStream для чтения/записи необработанных незакодированных двоичных данных
- QDataStream реализует сериализацию базовых типов данных
- Сериализация более сложных данных выполняется путем разделения данных на примитивные единицы

https://doc.qt.io/qtforpython-6/PySide6/QtCore/QDataStream.html

Демидовский университет

Класс QTextStream

- Строчка за строчкой
  - readLine() или readAll()
- Слово за словом в буферы QString, QByteArray и char*
  - Слова разделяются пробелом
- Посимвольно, путем потоковой передачи в типы QChar или char

https://doc.qt.io/qtforpython-6/PySide6/QtCore/QTextStream.html

Демидовский университет

Классы для работы с файлами и каталогами

- QFile — работа с файлами
- QFileInfo — информация о файле
- QDir — работа с директориями

Демидовский университет

QFile

- QFile — интерфейс ввода/вывода для текстовых и двоичных файлов
- open(), close() — открытие и закрытие файлов
- flush() — запись данных из буфера в файл без его закрытия
- setFileName() — задать имя файла после создания объекта
- exists() — проверка существования файла
- remove() — удаление файла
- Класс QTemporaryFile используется для безопасного создания уникальных временных файлов

Демидовский университет

https://doc.qt.io/qtforpython-6/PySide6/QtCore/QFile.html

QFileInfo

- Предоставляет информацию о свойствах файлов
  - имя
  - размер
  - время последнего изменения
  - права доступа и т. д.
- Объекта класса создается передачей в его конструктор пути к файлу или объекта класса QFile

https://doc.qt.io/qtforpython-6/PySide6/QtCore/QFileInfo.html

Демидовский университет

Пример работы с файлом

```
from PySide6.QtCore import QFile, QIODevice, QTextStream, QFileInfo

file = QFile("in.txt")
if not file.open(QIODevice.ReadOnly | QIODevice.Text):
    exit()
input_file = QTextStream(file)
while not input_file.atEnd():
    line = input_file.readLine()
    print(line)

info1 = QFileInfo("in.txt")
print(info1.isSymLink())
print(info1.absoluteFilePath())
print(info1.size())
```

Демидовский университет

QDir

- Работа с именами путей, доступом к информации о путях и файлах, базовой файловой системой
- « / » - универсальный разделитель каталогов
- Класс QDir может указать на файл по относительному или абсолютному пути
  - QDir("/home/user/Documents")
  - QDir("images/landscape.png")
- makeAbsolute() конвертирует относительный путь в абсолютный

https://doc.qt.io/qtforpython-6/PySide6/QtCore/QDir.html

Демидовский университет

Методы QDir

- path() — возвращает путь к каталогу
- setPath() — задает новый путь к каталогу
- dirName() — возвращает имя каталога
- cd() — метод для перемещения по каталогам
- cdUp() — эквивалент метода cd("..")
- mkdir(), rename(), rmdir()— методы для создания, переименования, удаления каталога

Демидовский университет

Методы QDir

- Статические методы для доступа к некоторым общим каталогам:
  - currentPath() — путь текущего каталога приложения
  - rootPath() — корневой каталог
  - tempPath() — временный каталог системы
  - homePath() — домашняя директория пользователя
- Следует использовать константы для доступа к каталогам, чтобы обеспечить кроссплатформенность
- В приложениях следует указывать относительные пути вместо абсолютных

Демидовский университет

QStandardPaths — стандартные пути к каталогам

- QStandardPaths.displayName(type). Типы:
  - QStandardPaths.DesktopLocation
  - QStandardPaths.HomeLocation
  - QStandardPaths.DocumentsLocation
  - QStandardPaths.MusicLocation
  - QStandardPaths.PicturesLocation
  - QStandardPaths.TempLocation
  - QStandardPaths.DownloadLocation
  - QStandardPaths.AppDataLocation
  - …

Демидовский университет

https://doc.qt.io/qtforpython-6/PySide6/QtCore/QStandardPaths.html

Диалог открытия файла – QFileDialog

- Статические функции
- getExistingDirectory(), getExistingDirectoryUrl() – выбрать каталог
- getOpenFileName(), getOpenFileNames() – выбрать файл для открытия
- getSaveFileName(), getSaveFileUrl() – выбрать файл для записи
- Можно фильтровать по расширениям

https://doc.qt.io/qtforpython-6/PySide6/QtWidgets/QFileDialog.html

Демидовский университет

Пример диалога

```
class Window(QMainWindow):
    def __init__(self):
        super().__init__()
        self.add_menu()
        layout = QVBoxLayout()
        self.label = QLabel()
        ...
    def add_menu(self):
        menuBar = self.menuBar()
        menuDialogs = menuBar.addMenu("Диалоги")
        actionFile = menuDialogs.addAction("Файл-картинка")
        actionFile.triggered.connect(self.clickActionFile)
    @Slot()
    def clickActionFile(self):
        fileName = QFileDialog.getOpenFileName(self, "Открыть картинку",
str(QDir.current()), "Картинки (*.png *.jpg)")
        print(fileName)
        self.label.setPixmap(QPixmap(fileName[0]))
```

емидовский
ниверситет

QFileSystemModel – модель файловой системы

- QFileSystemModel – наследник QAbstractItemModel
- Предоставляет некоторые удобные функции, специфичные для модели каталогов
- fileInfo(), isDir() , fileName(), filePath() – информация о базовых файлах и каталогах, связанных с элементами модели
- mkdir(), rmdir() – создать/удалить каталог
- setRootPath() – корневой каталог
  - QFileSystemModel не будет извлекать какие-либо файлы или каталоги до тех пор, пока не будет вызван setRootPath()

https://doc.qt.io/qtforpython-6/PySide6/QtWidgets/QFileSystemModel.html

Демидовский университет

Пример QFileSystemModel

```
class Window(QMainWindow):
    def __init__(self):
        super(Window, self).__init__()
        treeView = QTreeView()
        model = QFileSystemModel()
        model.setRootPath(str(QDir.home()))
        treeView.setRootIndex(model.index(str(QDir.home())))
        treeView.setModel(model)
        self.setCentralWidget(treeView)
```

Демидовский университет

QFileSystemWatcher – изменения в файловой системе

- QFileSystemWatcher отслеживает файловую систему на предмет изменений в файлах и каталогах, просматривая список указанных путей
- addPath(), addPaths() – добавить пути для отслеживания
- removePath(), removePaths() – удалить пути из списка
- QFileSystemWatcher проверяет каждый добавленный путь
- files(), directories() – доступ к файлам и каталогам, которые были добавлены
- fileChanged() выдается, когда файл был изменен, переименован или удален с диска
- directoryChanged() выдается, когда каталог или его содержимое изменяется или удаляется
- QFileSystemWatcher прекращает мониторинг файлов, как только они были переименованы или удалены с диска, и каталогов, как только они были удалены с диска

https://doc.qt.io/qtforpython-6/PySide6/QtCore/QFileSystemWatcher.html

QFileSelector – файлы по шаблону

- Доступные селекторы:
  - платформа, на которой запущено приложение (список не является исчерпывающим): android, ios, osx, darwin, mac, macos, linux, qnx, unix, windows
  - В Linux также указывается название дистрибутива, например debian, fedora
  - языковой стандарт, такой же, как QLocale().name()
- selector = QFileSelector()
- defaultsFile = QFile(selector.select("data/defaults.conf"))
  - data/defaults.conf
  - data/android/defaults.conf
  - data/ios/en_GB/defaults.conf
- setExtraSelectors(list) – добавить селекторы

https://doc.qt.io/qtforpython-6/PySide6/QtCore/QFileSelector.html

Демидовский университет

Работа с файлами в приложении

- Работа с данными из файлов осуществляется в модели
- Если пользователю нужно выбрать файл, нужно использовать один из стандартных диалогов
  - Не следует просить пользователя вводить полностью путь к файлу
- Если файл заранее известен, данные из него загружаются сразу
- В связке Qt + Python обработка файлов в модели может осуществляться не только средствами Qt, но и с помощью встроенных функций и дополнительных библиотек
  - Нужно выбирать технологию, наиболее подходящую для решения задачи